

CHAPTER I

Introduction & Background

1.1. QRIS Adoption

QRIS, Indonesia's national QR payment standard, was implemented by Bank Indonesia via Payment Administrative Order No. 21/18/PADG/2019 and developed in collaboration with the Indonesian Payment System Association (ASPI). Based on the EMVCo QR code specification for payment systems, it is fully interoperable: a single code displayed by a street vendor can be used for payments via any authorized mobile banking app or e-wallet. Its adoption has been massive. By the end of 2025, QRIS recorded the highest growth in transaction volume of any payment system in Indonesia, rising 139.9% year-on-year and reached 59.53 million users and 42.75 million merchants (Bank Indonesia, 2026) and as of mid 2025, 93.16% of its merchants are SMEs (Bank Indonesia, 2025). QRIS has become the backbone of retail digital payments in Indonesia, particularly for small businesses.

1.2. Physical Attack Surface

This extensive reach also creates an asymmetric physical attack surface. The QRIS (Indonesian Quick Response Code Standard) displays used by most micro, small, and medium-sized enterprises (MSMEs) typically consist of static QR codes printed on inexpensive paper, laminated cards, or acrylic stands. Unlike Point-of-Sale (POS) terminals, these displays serve merely as passive data carriers, while they direct payments to the merchant's account, they lack electronic displays, challenge-response mechanisms, or tamper-detection capabilities, nor can they verify that the QR code actually belongs to the specific stall where it is displayed. The risk is compounded by where they are used. In public markets, food courts, roadside stalls, and at donation boxes, these QR codes are often left unattended and easily accessible to malicious actors, making them highly vulnerable to undetected tampering or replacement.

1.3. The Sticker-Swap Attack

In a "sticker swap" attack, the attacker does not actually forge anything. Instead, they register a legitimate merchant account with an authorized payment service provider (PJP), obtain an officially issued QR code (compliant with the Indonesian standard), and then place it over the original merchant's QR code. This causes all payments to bypass the legitimate merchant entirely and go directly to the attacker. While the media refers to these as "fake QR codes," the codes themselves are genuine, only their placement is deceptive. This type of attack gained nationwide attention in April 2023, when an individual placed such stickers at 38 mosques and prayer rooms across Jakarta and Tangerang, generating illicit proceeds of 13.06 million Indonesian rupiah between April 1 and April 9 (DW, 2023). These QR codes, labeled "Restorasi Masjid", were linked to accounts registered in Medan and South Jakarta (Kompas, 2023). Since then, this type of attack has spread to everyday business operations. In late September 2025, vendors at a food court at Telkom University in the Bandung area discovered that the QR codes for approximately 10 to 20 stalls had been covered over. Each stall suffered losses ranging from 30,000 to 1 million rupiah, and it is believed the stickers were applied overnight when the area was deserted (Kompas TV, 2025).

1.4. Why Inspecting the Code Is Not Enough

To determine whether legitimate and malicious QR code tags could be distinguished solely through payload analysis, Q-Shield compared 16 structural and semantic attributes extracted from legitimate and fraudulent QR code payloads. However, none of these attributes could differentiate between the two types of tags. The only visible difference, a payload length of 152 versus 154 characters, comes from the length of the merchant name and varies among genuine merchants as well. This is expected: the attacker does not forge signatures or alter the payload but exploits the payment system's lack of spatial context. The fraud is not in the code, it is in the code's physical placement. Detecting it therefore requires the one thing QRIS does not have knowledge of where a code is supposed to be.

CHAPTER II

Solution Overview & Market Differentiation (USP Analysis)

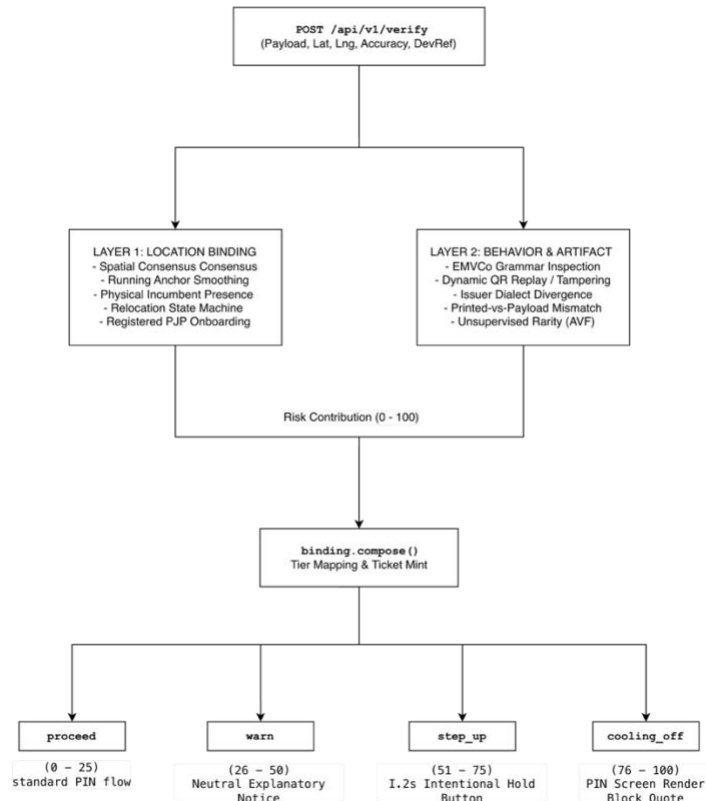
2.1. Q-Shield Solution Overview

Q-Shield introduces an inline, low-latency, zero-trust verification middleware that intercepts the payment execution flow before the transaction becomes irreversible:

Camera Feed→Payload Extraction→POST /api/v1/verify→Dynamic Friction Gate→PIN Entry

Rather than relying on post-transaction fraud detection, Q-Shield runs synchronous pre-PIN assertions using a Two-Layer Composite Architecture:

- **Layer 1 (Merchant-Location Binding):** Determines whether the scanned National Merchant ID (NMID) is established and verified at the user's geographic coordinates via distributed observer consensus.
- **Layer 2 (Artifact & Structural Behavioral Scoring):** Determines whether the payload exhibits abnormal syntactic dialects, dynamic QR re-use, invoice amount modification, or printed-vs-encoded mismatches.



CHAPTER III

Proof of Concept (PoC) Implementation & Repository References

3.1. Repository Structure and Modular Design

The Q-Shield code is organized as a modifiable package under *src/qshield/*, separating the evaluation logic from persistence and network I/O. The data analysis (payload) and spatial queries (spatial queries) are initiated via *emvco.py* (recursive analysis of EMVCo TLV and validation of CRC16) and *geo.py* (geohash-7 indexing with extended haversine distance checks). These feed the data into the two-layer verification pipeline: *binding.py* evaluates the spatial consensus on the first layer (layer 1) and the smoothing of the center of gravity of the anchor points, while *behavior.py* and *profile.py* enforce rules for the grammar of the second layer (layer 2), matched with emitter dialects, and unsupervised detection of categorical outliers (AVF). The system state is ensured in *store.py* via a thread-safe SQLite WAL engine that ensures strict separation of personal data (Zero-PII), while *ticket.py* creates short-lived HMAC-SHA256 verification tokens that are coordinated via FastAPI (*api.py*) and a dual-mode mobile web scanner (*web/index.html*).

Operational thresholds and resilience guarantees are based on calibration in dedicated test suites. The *scripts/* directory stores empirical operational parameters, including spatial search thresholds *calibrate_geo.py*, false positive thresholds based on 20,000 synthetic loads *calibrate_layer2.py*, convergence tests of anchor points, stress tests and trace flight, and preflight control *preflight.py*. The test systems guarantee system reliability through regression checks validating eight architectural security invariants *test_invariants.py* and 44 simulated adversarial attack scenarios *test_adversarial.py*, while counter settings are verified against rat/SDK mechanisms provide an isolated integration interface for native mobile clients.

3.2. Core Component Implementation Breakdown

1. Spatial Ingestion and Consensus Engine (Layer 1) - *emvco.py*, *geo.py*, *binding.py*

What it Extracts structured merchant data (such as the National Merchant ID / NMID) from the raw QR code and cross-references it with the customer's physical GPS

coordinates. Instead of relying on a single reading, it aggregates scans from various independent shoppers to build an "anchor" of trusted merchants. The engine continuously refines the store's precise map location over time, handles the movement or relocation of food trucks, and implements physical presence checks so that legitimate neighboring merchants (such as adjacent stalls in a food court) are not falsely flagged as fraudulent.

2. Artifact and Behavioral Anomaly Engine (Layer 2) - *behavior.py, profile.py*

Inspects the digital structure of the QR code itself for signs of tampering, even if the store is new. It validates mandatory payment syntax, detects fraud-generating tools by comparing payload formats against known banking conventions (issuer dialects), and identifies instances where single-payment codes (dynamic QRs) are reused at different locations or have their transaction amounts altered. It also employs unsupervised pattern profiling to flag suspicious merchant attribute combinations and verifies that the physical text printed at the point of sale matches the code embedded within it.

3. Persistence and Zero-PII Cryptography Gateway - *store.py, ticket.py, api.py*

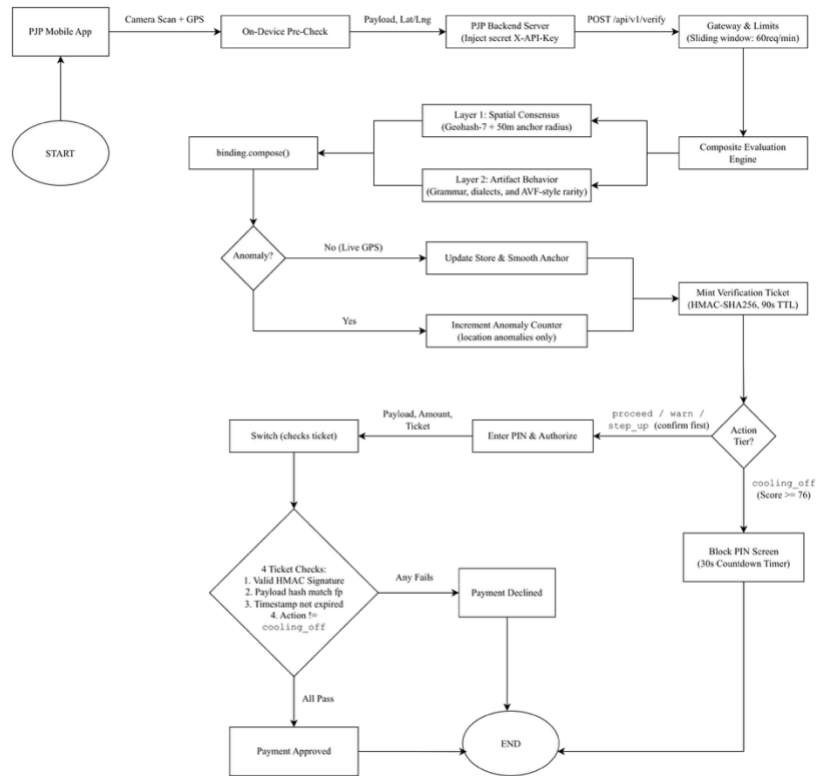
Handles high-speed data storage and enhances security without tracking user behavior. The database stores no personal customer information (no names, phone numbers, or user GPS traces) and isolates device IDs using hashing combined with location data to ensure full compliance with data protection laws (UU PDP). Once a scan passes verification, the engine generates a short-lived (90-second), tamper-proof digital stamp known as a verification ticket. This cryptographically links the security verdict to the specific scanned QR code, ensuring that malware or malicious apps cannot verify a secure QR code and subsequently surreptitiously swap in a fraudster's payment details during the transaction.

CHAPTER IV

Technical Architecture & Feasibility

4.1. System Flowchart

When a client scans a QRIS code, the mobile app captures the data (payload) and GPS location and routes it via the PJP backend, which attaches the API Key and invokes the Q-Shield POST `/api/v1/verify` endpoint. The system evaluates the scan at two levels: Layer 1 (spatial consensus within a 50 m radius based on Geohash-7) and Layer 2 (payload structure, issuer dialects and AVF-style rarity). If an anomaly is detected, only the anomaly count for that location is incremented. Otherwise, the merchant anchor point is updated based on the scan data. In either case, Q-Shield returns the result along with an HMAC-SHA256 signed verification ticket valid for 90 seconds. If the status is set to *cooling_off*, the app blocks the PIN entry interface and displays a 30-second countdown. While other operations permit PIN entry, the *step_up* operation requires prior confirmation. Finally, the switch checks the ticket's signature, payload fingerprint, expiry, and action, approving the payment only if all four checks pass and declining it otherwise.



CHAPTER V

Security Architecture & Intellectual Property Potential

5.1. Threat Matrix & Empirical Mitigations (STRIDE Alignment)

ID	Threat Vector	Impact	Defense Mechanism	Verification Suite
T1	Physical Sticker Overlay	High	Layer 1 spatial consensus; $W \geq 83 \rightarrow$ cooling_off	tests/test_binding.py
T2	Sybil Cold-Start Flooding (<5 attacker devices)	High	Consensus requires ≥ 3 dev over 24h; ratio threshold	tests/test_adversarial.py
T3	Dynamic QR Replay / Invoice Tampering	Med	Tag 62 bill tracking and amount caching (48h TTL)	tests/test_emvco.py
T4	Client-Side TOCTOU Bypass	High	Stateless HMAC-SHA256 Verification Ticket	tests/test_ticket.py
T5	Database Compromise / Tracking	High	Zero-PII schema; scoped device_ref hashing	tests/test_invariants.py
T6	OS-Level Mock Location Spoof	Med	Accuracy plausibility gate (<1.0m flagged) + AppAttest	tests/test_hardening.py

5.2. Cryptographic Verification Ticket Architecture

Exploitation of security vulnerabilities could lead to a "Time-of-Check to Time-of-Use" (TOCTOU) attack, where a different QR code is used for payment during the actual transaction process after authorization based on a valid QR code has been obtained. To prevent such attacks, all Q-Shield detection results include the SHA-256 fingerprint of the

scanned QR code and the detection outcome, accompanied by a ticket signed using HMAC-SHA256 (valid for 90 seconds). Prior to payment, the PJP approves the transaction only if:

$$\text{SHA-256}(P_{\text{pay}}) = T.fp \wedge \text{HMAC-Verify}(K_{\text{PJP}}, T) \wedge \text{Now} < T.exp \wedge T.action \in A_{\text{allowed}}$$

The tampered QR code failed fingerprint authentication, and the ticket displaying a "payment denied" result failed final authentication. Although the ticket linked the authentication result to the payment operation, it could not force the PJP (payment terminal) to execute that result, allowing the ticket to be reused within its 90-second validity period.

5.3. Intellectual Property (IP) & Patent Potential

The technical mechanisms developed in Q-Shield represent novel, non-obvious engineering solutions:

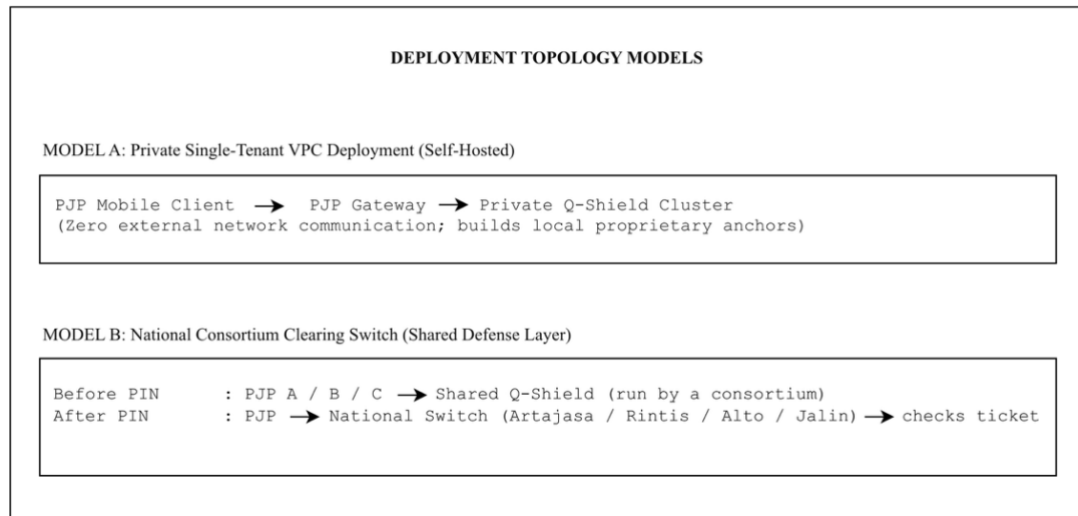
Invention Title	Technical Novelty & Core Method Claims
Method 1: Multi-Entity Spatial Consensus for Offline Payment Terminal Verification	A method of crowdsourcing trusted merchant anchors across disparate payment service providers using bounded running average coordinate convergence and location-scoped hashing.
Method 2: Physical Incumbent Presence Tracking for Countering Overlay Fraud	An automated mechanism resolving physical terminal coexistence conflicts without manual onboarding by verifying historical scan continuity of incumbent terminals during challenger entry.
Method 3: Decoupled Pre-PIN Verification Ticket Protocol	A secure transaction handshake protocol generating an in-flight HMAC-SHA256 token that cryptographically binds pre-PIN risk verdicts to raw settlement payload digests under short TTLs.

CHAPTER VI

Scalability & Deployment Readiness

6.1. Enterprise Deployment Topologies

Q-Shield supports two operational models depending on the PJP's regulatory and architectural requirements:



In Model A, a single PJP runs Q-Shield privately inside its own infrastructure: its app sends each scan to its own backend, and no data leaves the PJP, but the merchant map is built only from that PJP's own users. In Model B, all PJPs query one shared Q-Shield run by a consortium before the PIN screen. It keeps a single map of merchant locations built from every app's scans, containing no personal data, so a scan by one app's user protects every other app's users. After the PIN, the national switch checks the verification ticket issued in the first step before the payment proceeds. A PJP can start with Model A and join Model B later.

6.2. Native Mobile SDK Seam & Client Integration

The integration boundary for Android (Kotlin) clients is defined in *sdk/README.md* and backed by contract fixtures in *sdk/contract/*:

1. **On-Device Pre-Parsing:** The client SDK executes initial EMVCo TLV parsing directly in native memory. Non-EMVCo QR codes (e.g., standard URLs, phishing links) are rejected locally without initiating an HTTP roundtrip.
2. **Device Attestation Slot:** The client gathers hardware attestation metrics via Google Play Integrity API or Apple App Attest, populating the *device_integrity* payload:

```
{
  "mock_location": false,
  "rooted": false,
  "attested": true,
  "platform": "android"
}
```

3. **Fail-Safe Offline Mode:** If a network timeout exceeds 200ms, the SDK falls back gracefully to warn, displaying an informational banner reminding the customer to verify the store's physical signage.

6.3. The Dual-Mode Web Scanner

The web scanner (*src/qshield/web/index.html*) includes multi-camera selection logic that steers away from the ultra-wide (0.5x) lens and prioritizes the 1x main camera with continuous autofocus. This ensures QR codes are read reliably in both of the scanner's modes, which can be switched at runtime to support consumer-facing verification and backend merchant auditing:

- **Verification Mode (POST /api/v1/verify):** Captures real-time GPS coordinates and emits full risk scoring.
- **Catalog Mode (POST /api/v1/inspect):** Dispatches raw payloads to the inspection endpoint without location coordinates.